

Tuple Relational Calculus (TRC)

1. What is Tuple Relational Calculus?

Tuple Relational Calculus (TRC) is a non-procedural / declarative query language based on relational calculus [cite: topic-13]. Rather than providing step-by-step instructions on how to evaluate a query, TRC specifies **what** data is required by establishing conditions that tuples must satisfy [cite: topic-13].

Feature	Relational Algebra	Tuple Relational Calculus (TRC)
Type	Procedural [cite: topic-13]	Non-procedural / Declarative [cite: topic-13]
Core Focus	Describes operations (HOW to obtain result) [cite: topic-13]	Describes conditions (WHAT data is required) [cite: topic-13]
Primary Mechanism	Uses algebraic operators (σ , π , $\Join$, $\cup$, $-$) [cite: topic-13]	Uses tuple variables and predicates ($P(t)$) [cite: topic-13]

2 & 3. Tuple Variables & Basic Syntax

A **tuple variable** represents a tuple (row) of a relation schema [cite: topic-13]. The general notation for a TRC expression is [cite: topic-13]:

$$\{ t \mid P(t) \}$$
 — "Set of all tuples t such that predicate $P(t)$ is true" [cite: topic-13]

To specify that tuple s belongs to relation *Supplier* with a condition on its address [cite: topic-13]:

$$\{ s \mid s \in \text{Supplier AND } s.\text{Address} = \text{'Chandigarh'} \}$$
 [cite: topic-13]

 **Golden Rule:** TRC returns only those tuples that evaluate the predicate condition to TRUE [cite: topic-13].

4 & 5. Predicates & Logical Operators

A **predicate** is a boolean expression evaluating to true or false for each tuple [cite: topic-13]. TRC connects predicates using logical operators [cite: topic-13]:

- **AND ($P \wedge Q$):** Both conditions must hold true [cite: topic-13].
- **OR ($P \vee Q$):** At least one condition must hold true [cite: topic-13].
- **NOT ($\neg P$):** Negates the predicate evaluation [cite: topic-13].

6–8. Quantifiers ($\exists$ and $\forall$)

Quantifiers state condition scopes over whole sets of tuples [cite: topic-13]:

Quantifier	Name / Reading	Query Keywords	Meaning
$\exists$	Existential ("There exists") [cite: topic-13]	SOME, ANY, AT LEAST ONE [cite: topic-13]	At least one tuple satisfies the condition [cite: topic-13].
$\forall$	Universal ("For all") [cite: topic-13]	EVERY, ALL, EACH [cite: topic-13]	The condition must hold for every relevant tuple [cite: topic-13].

Existential Example: "Suppliers who supply at least one red part." [cite: topic-13]

$$\{ s \mid s \in \text{Supplier AND } \exists p \in \text{Part } (s.\text{SID} = p.\text{SID AND } p.\text{Color} = \text{'Red'}) \}$$
 [cite: topic-13]

Universal Example: "Suppliers who supply every part." [cite: topic-13]

$$\{ s \mid s \in \text{Supplier AND } \forall p \in \text{Part } (\text{condition}) \}$$
 [cite: topic-13]

9. Free and Bound Variables

Understanding variable scopes in TRC formulas [cite: topic-13]:

- **Free Variable:** A tuple variable that is NOT bound by any quantifier [cite: topic-13]. It appears on the left side of the bar (e.g., s in $\{ s \mid P(s) \}$) and represents the target output tuples [cite: topic-13].
- **Bound Variable:** A variable that falls within the scope of an existential ($\exists$) or universal ($\forall$) quantifier (e.g., p in $\exists p \in \text{Part}$) [cite: topic-13].

🏆 MEMORY TRICK

Free Variable → Appears in the final query result [cite: topic-13]. | **Bound Variable** → Controlled internally by a quantifier ($\exists$ or $\forall$) [cite: topic-13].

10 & 11. Worked Example & RA vs TRC Comparison

Sample Database Tables [cite: topic-13]

Supplier:

SID	SName	Address
S1	A	Delhi
S2	B	Chandigarh

Query: "Find supplier names whose address is Chandigarh." [cite: topic-13]

TRC Solution: [cite: topic-13]

```
{ s.SName | s ∈ Supplier AND s.Address = 'Chandigarh' } [cite: topic-13]
```

Relational Algebra Solution: [cite: topic-13]

```
 $\pi_{SName} ( \sigma_{Address='Chandigarh'} (Supplier) )$  [cite: topic-13]
```

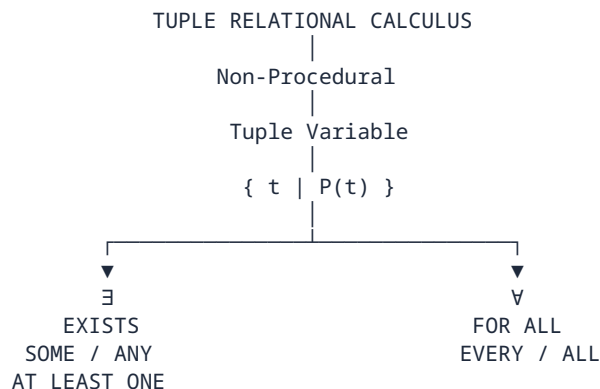
Result: B [cite: topic-13]

12. Safe vs Unsafe Expressions ⚠️

Safety guarantees that query evaluation produces predictable, finite results [cite: topic-13]:

- **Safe Expression:** A TRC expression whose result is strictly finite and composed only of domain values present in the database relations or explicit constants [cite: topic-13].
- **Unsafe Expression:** An expression that can return an infinite or unbounded number of tuples (e.g., $\{ t \mid \neg(t \in \text{Supplier}) \}$) [cite: topic-13]. Unsafe queries are prohibited in relational systems [cite: topic-13].

13. TRC Structural Architecture Summary



⚠️ High-Frequency Exam Traps

COMMON EXAMINATION PITFALLS TO AVOID:

- **Trap 1: Classifying TRC as Procedural:** TRC is strictly non-procedural/declarative (specifies WHAT, not HOW) [cite: topic-13].
- **Trap 2: Confusing Quantifiers:** $\exists$ means "at least one", whereas $\forall$ means "every / all" [cite: topic-13].
- **Trap 3: Misidentifying Variable Types:** Free variables define output attributes; bound variables are localized inside quantifiers [cite: topic-13].
- **Trap 4: Expecting Sequence Steps:** TRC does not define execution steps or relational operation ordering [cite: topic-13].

🏆 ULTIMATE MUST-REMEMBER SUMMARY

- **TRC Formula:** $\{ t \mid P(t) \}$ [cite: topic-13]
- **TRC Focus:** WHAT data is required [cite: topic-13] | **Relational Algebra Focus:** HOW to obtain the data [cite: topic-13]
- $\exists$: Exists / Some / Any / At least one [cite: topic-13] | $\forall$: For all / Every / All [cite: topic-13]
- **Free variable:** Result variable [cite: topic-13] | **Bound variable:** Quantifier-controlled [cite: topic-13]
- **Safe Expression:** Yields finite, database-related output [cite: topic-13]